

Step 4.2 – Sales-Inventory Settlement Synchronization Implementation Plan

Tujuan & Latar Belakang

Step 4.2 adalah bagian dari MVP fase 4 yang bertujuan memformalkan settlement pembayaran dalam domain penjualan tanpa merusak domain lain. Dokumen ini merangkum rancangan teknis, pre-condition, non-goals, dan rencana implementasi terperinci.

Tujuan Utama

- **Derived outstanding:** Outstanding tidak lagi disimpan sebagai nilai mutable, melainkan dihitung sebagai `totalAmount - sum(payments)`.
- **Partial payment:** Sistem harus mengakomodasi pembayaran bertahap dan memastikan total pembayaran tidak melebihi total order.
- **Concurrency safety:** Settlement harus berjalan dalam satu transaction dengan optimistic locking melalui field `Order.version`.

Pre-Conditions

Implementasi Step 4.2 hanya boleh dimulai jika: 1. Step 4.1 (Stock Origin) telah selesai dan seluruh test lulus. 2. Reporting Step 3 tetap deterministik. 3. Tidak ada perubahan write-model lain di luar scope dokumen ini.

Selain itu, Step 4.2 harus mematuhi constraint dari Step 4.1 close-out. Dokumen close-out menegaskan bahwa Step 4.2 boleh dimulai hanya jika tidak mengubah logic `origin`, tidak mengubah pola mutasi inventory, dan tidak melakukan redesign origin. Dengan kata lain, integrasi settlement pada Step 4.2 harus menjaga boundary Inventory dan origin yang telah diaktifkan.

Non-Goals

Step 4.2 tidak mencakup perubahan pada: - Inventory Domain - Reporting - Sistem akuntansi - Multi-currency

Perubahan harus menjaga pattern mutasi inventory, batas domain, dan desain origin stock.

Scope Implementasi

Entitas Payment

Desain mensyaratkan entitas `Payment` dengan field `id`, `orderId`, `amount`, `paidAt`, `method`, dan `createdAt`, serta bersifat immutable.

Beberapa klarifikasi penting untuk memastikan konsistensi implementasi:

- `method` **sebagai string bebas**: kolom `method` tidak dibatasi oleh enum tertentu. Nilainya dapat berupa kode jenis pembayaran apa pun (misalnya `"CASH"`, `"TRANSFER"` atau metode lainnya) dan dapat dievolusi di masa depan tanpa perubahan schema.
- **Perbedaan `paidAt` dan `createdAt`**: `paidAt` merepresentasikan waktu terjadinya transaksi pembayaran di lapangan, sedangkan `createdAt` merekam kapan fakta pembayaran dicatat ke database. Kedua field harus dipertahankan untuk keperluan audit trail dan akuntansi.

Pada kode sekarang, entitas `Payment` sudah ada dengan `id`, `orderId`, `amount`, `occurredAt`, dan `createdAt`.

Implementasi harus menambahkan: - Field `paidAt` (mengganti `occurredAt`) untuk mencatat waktu pembayaran, serta field `method`. Field `createdAt` tetap dipertahankan sebagai timestamp pencatatan. - Constructor yang memaksa nilai immutable; koreksi pembayaran dilakukan dengan membuat `Payment` baru - Validasi untuk mencegah `amount <= 0`; kalau invalid lempar `InvalidPaymentAmountError`

Derived Outstanding

Saat ini `Order` memiliki property `totalAmount` dan `outstandingAmount` serta method `recomputeOutstanding`.

Implementasi harus memastikan: - `outstandingAmount` tidak di-mutasi langsung; dihitung ulang dari histori payment setiap settlement - Pada state transisional, `outstandingAmount` tetap disimpan sebagai cache fisik di database namun bukan source of truth

Concurrency & Optimistic Locking

- Tambahkan field `version` (default `0`) pada tabel `Order`
- Optimistic locking diterapkan dengan menambah parameter `expectedVersion`; update hanya berhasil jika `version` di database sama dengan `expectedVersion`, kemudian versi di-increment
- Jika update count `0`, lempar `OptimisticLockConflictError`
- Settlement harus dibungkus dalam satu database transaction dengan langkah:
 - load `Order`
 - validasi status
 - hitung total payment
 - validasi over-payment
 - persist `Payment`

- hitung ulang outstanding
- simpan `Order` dengan version check
- Implementasikan retry maksimal dua kali jika terjadi conflict versi

Rencana Implementasi Berurutan

1. Migrasi Database & Schema

1. Tambahkan kolom `version` pada tabel `Order` dengan default `0`
2. Perluas tabel `Payment` dengan kolom `method` (`VARCHAR`), rename/alias `occurredAt` menjadi `paidAt`, dan pastikan kolom `createdAt` tetap ada sebagai timestamp pencatatan. `paidAt` digunakan untuk merekam waktu kejadian transaksi pembayaran, sedangkan `createdAt` merekam waktu entri data.
3. Pastikan semua perubahan skema kompatibel dengan engine **MySQL** yang digunakan (misalnya, gunakan tipe `INT` dengan default `0` untuk kolom `version`).
4. Buat index untuk `orderId` dan `paidAt`
5. Pastikan migrasi additive untuk menjaga kompatibilitas existing data

2. Pembaharuan Domain

Order

- Tambahkan property `version` (private) dengan getter
- Masukkan ke parameter factory dan reconstitution
- Jangan ubah invariants lain

Payment

- Perbarui konstruktor agar menerima `paidAt` dan `method`
- Ganti pemakaian `occurredAt` menjadi `paidAt`
- `Payment` harus immutable

Error Classes

Tambahkan kelas error berikut sesuai guideline error-handling: - `PaymentOverpayError` - `InvalidPaymentAmountError` - `OrderNotOnCreditError` - `OptimisticLockConflictError`

3. Repository Layer

OrderRepository

Ganti interface menjadi:

```
findById(id: EntityId, tx?: Transaction): Promise<Order | null>
saveWithVersionCheck(order: Order, expectedVersion: number, tx: Transaction):
Promise<void>
```

Implementasikan `PrismaOrderRepository` : - gunakan prisma transaction (`prisma.$transaction`) - lakukan update version atomik dengan pola:

```
where: { id, version: expectedVersion }  
data: { ..., version: { increment: 1 } }
```

PaymentRepository

- Tambahkan parameter optional transaction pada `save` dan `sumAmountByOrderId`
- Pastikan query `sumAmountByOrderId` terjadi di dalam `tx`
- Implementasikan `PrismaPaymentRepository` yang benar, termasuk menyimpan kolom `method` dan `paidAt`

4. Use-Case PayCredit

Transaction Boundary

Bungkus seluruh settlement di dalam callback transaction dari repository, misalnya `prisma.$transaction`.

Langkah Algoritma

Ikuti pseudocode design: 1. load order + versi 2. cek status `ON_CREDIT` 3. validasi `amount > 0` dan tidak melebihi outstanding 4. hitung `totalPaid` dari histori 5. buat `Payment` 6. hitung `totalPaidAfter` 7. panggil `order.recomputeOutstanding` 8. simpan order via `saveWithVersionCheck`

Retry

- Implementasikan loop untuk menangani `OptimisticLockConflictError`
- Maksimal dua percobaan

Error Handling

- Gunakan error khusus
- Hindari `new Error` generik

Pengembalian Status

- Setelah payment sukses, periksa apakah outstanding menjadi nol
- Jika ya, status order berubah menjadi `PAID`
- Jika tidak, tetap `ON_CREDIT`

5. Testing & Hardening

Unit Test

Tambahkan test untuk `Payment` dan `Order` terkait version increment dan error. Pastikan hal berikut ter-handle: - partial payment - over-payment - invalid amount

Integration Test - Race Condition

- Jalankan dua `PayCredit` secara paralel pada order yang sama
- Verifikasi salah satu eksekusi gagal atau retry
- Pastikan tidak ada overpayment

Integration Test - Double Submit

- Tanpa idempotency key, jalankan dua request dengan amount sama secara paralel
- Pastikan `Payment` hanya tersimpan sekali
- Pastikan overpayment tidak terjadi

Reporting Regression

- Jalankan ulang suite reporting Step 3
- Pastikan penambahan `Payment` dan `version` tidak mengubah determinisme laporan

Inventory & Origin Protection

- Pastikan tidak ada call baru ke modul Inventory
- Pattern mutasi stok tetap mengikuti panduan existing
- Origin stock tidak boleh berubah

Catatan & Pengawasan

- **Dokumentasi:** Setiap perubahan harus disertai log note sesuai panduan `log_note_writing_guidelines.md` dan commit message yang jelas
- **Clean code & DDD:** Jaga konsistensi dengan arsitektur DDD, modul boundaries, dan guideline penulisan kode; hindari leakage domain antar layer
- **Audit trail:** Pastikan transaksi settlement tercatat sesuai `audit-trail-policy.md`

Kesimpulan

Dengan mengikuti rencana ini, implementasi Step 4.2 dapat dilakukan secara terkontrol dan terukur. Perubahan utama terletak pada: - penambahan kolom `version` pada `Order` - pengayaan entitas `Payment` - pengenalan transaksi dan optimistic locking dalam use-case `PayCredit` - penambahan error handling dan test konkruen

Seluruh proses harus tetap menjaga batas domain Inventory dan reporting, serta mempertahankan prinsip clean code dan DDD.

Referensi File yang Disebut

- `Payment.ts`
 - `Order.ts`
-